2023013182 华宜佳

```
import numpy as np

def banker_algorithm(max_matrix, need_maxtrix, available_vector, allocated_matrix,
                     request_vector, process_id):

    number
    num_processes = max_matrix.shape[0]
    num_resources = max_matrix.shape[1]

    #1.检查 Request <= Need
    if not np.all(request_vector <= need_matrix[process_id]):
        print(f"错误：进程{process_id}申请的资源超过了声明的需求。")
        return False, None, None, None
    #2.检查 Request <= Available
    if not np.all(request_vector <= available_vector):
        print(f"错误：资源不足，进程{process_id}申请的资源超过了当前可用资源。")
        return False, None, None, None
    #3.尝试分配资源（假设分配）
    temp_available = available_vector - request_vector
    temp_allocation = allocated_matrix.copy()
    temp_allocation[process_id] += request_vector
    temp_need = need_matrix.copy()
    temp_need[process_id] -= request_vector
    #4.安全性检查
    finish = np.zeros(num_processes, dtype=bool)
    Work = temp_available.copy()
    safe_sequence = []

    while True:
        found = false
        for i in range(num_processes):
            if not finish[i]:
                if np.all(temp_need[i] <= work):
                    work += temp_allocation[i]
                    finish[i] = True
                    safe_sequence.append(i)
                    found = True
```

```
            if not found:
                break

        if np.all(finish):
            print(f"安全：资源可以分配给进程{process_id}。安全序列：{safe_sequence}")
            return True, temp_need, temp_available, temp_allocation
        else:
            print(f"危险：分配给进程{process_id}后系统将进入不安全状态")
            return False, None, None, None

if __name__ == "__main__":
    #资源数量
    num_processes = 5
    num_resources = 3

    #初始化矩阵（使用Numpy数组）
    max_matrix = np.array([
        [7, 5, 3],
        [3, 2, 2],
        [9, 0, 2],
        [2, 2, 2],
        [4, 3, 3]
    ])
    allocated_matrix = np.array([
        [0, 1, 0],
        [2, 0, 0],
        [3, 0, 2],
        [2, 1, 1],
        [0, 0, 2]
    ])
    available_vector = np.array([3, 3, 2])
    #计算 Need 矩阵
    need_matrix = max_matrix - allocated_matrix
    print("初始状态")
    print("Max:\n", max_matrix)
    print("Allocation:\n", allocated_matrix)
    print("Need:\n", need_matrix)
    print("Available:", available_vector)
    print("\n" + "=" * 50 + "\n")
```

```
# 测试用例 1：安全分配
request_vector1 = np.array([1, 0, 2])
process_id1 = 1
print(f"测试 1：进程{process_id1}申请资源{request_vector1}")
is_safe1, new_need1, new_available1, new_allocation1 = banker_algorithm(
    max_matrix, need_matrix, available_vector, allocation_matrix, request_vector1,
    process_id1)
if is_safe1:
    print("\n分配后状态：")
    print("Allocation:\n", new_allocation1)
    print("Need:\n", new_need1)
    print("Available:", new_available1)
else
    print("资源分配被拒绝")
print("\n" + "=" * 50 + "\n")

# 测试用例2：不安全分配
request_vector2 = np.array([3, 3, 0])
process_id2 = 4
print(f"测试 2：进程{process_id2}申请资源{request_vector2}")
is_safe2, _, _, _ = banker_algorithm(
    max_matrix, need_maxrix, available_vector, allocated_matrix, request_vector
    process_id2)
print("\n" + "=" * 50 + "\n")

# 测试用例3：超过需求
request_vector3 = np.array([1,2,0])
process_id3 = 0
print(f"测试3：进程{process_id3}申请资源{request_vector3}")
is_safe3, _, _, _ = banker_algorithm(
    max_matrix, need_matrix, available_vector, allocated_matrix, request_vecto
    process_id3)
```